

DSA Pattern Handbook

Complete Interview Preparation Guide — Beginner to Advanced

How to use this handbook: Read each section once slowly to build intuition, then use it as a revision reference. Every table row is a mental model, not a formula to memorize. The goal is to *recognize* patterns from problem phrasing, not recall solutions.

SECTION 1: ARRAY PATTERNS

Arrays are the foundation of DSA. Most interview problems involve arrays in some form. The patterns below apply to contiguous memory sequences (arrays and strings are interchangeable in most of these).

Two Pointers

Two Pointers is the art of maintaining two indices simultaneously to reduce $O(n^2)$ brute-force scans to $O(n)$. The key insight is: if you know something about the relative order of elements (sorted, or a structural property), two pointers let you eliminate entire ranges of candidates in one step.

Opposite Ends

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Two Pointers	Opposite Ends	Start one pointer at index 0 and one at index $n-1$; move them toward each other based on a condition	"sorted array", "pair that sums to target", "container with most water"	Sorted array, looking for a pair satisfying a condition	If sum too large $\rightarrow$ move right pointer left; if sum too small $\rightarrow$ move left pointer right	$O(n)$ time, $O(1)$ space	Requires sorted order or a clear monotonic property	Three-sum (fix one, apply this on rest)

Same Direction (Slow-Fast)

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Two Pointers	Same Direction	Both pointers move left-to-right; slow pointer marks a "write" position, fast pointer explores	"remove duplicates", "move zeros", "in-place filter"	In-place array modification without extra space	Fast pointer scans; when a valid element is found, write it at slow pointer position, then advance slow	O(n), O(1)	Logic can be subtle to get right	Overwrite vs swap

Fast-Slow Pointer

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variant
Two Pointers	Fast-Slow (Floyd's)	Slow moves 1 step, fast moves 2 steps; they meet iff a cycle exists	"cycle", "loop", "middle of list", "repeated number"	Cycle detection, finding midpoint	If they ever meet → cycle exists; meeting point used to find cycle start	Detects cycle in O(n) time and O(1) space	Only works on structures with a "next" notion	Find cycle start, find duplicate number

Partition Pointer

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variant
Two Pointers	Partition Pointer	One pointer marks the boundary between two regions as you scan	"partition array", "separate positive/negative", "sort colors"	In-place partitioning into two groups	Maintain a boundary index; everything to its left satisfies condition A, right side is unprocessed	O(n), O(1)	Two-region only; three regions needs Dutch Flag	Low vs partition

Three Pointer

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variant
Two Pointers	Three Pointer	Fix one pointer, apply two-pointer technique on the remaining range	"three numbers", "triplets", "3Sum"	Finding triplets satisfying a condition	Sort array; for each fixed element, run opposite-ends two pointer on the rest	O(n²) which is optimal for 3Sum	Duplicates must be carefully skipped	3Sum Closest, 4Sum

Dutch National Flag

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Two	Dutch	Three-way	"three	Sorting/partitioning	low tracks	Single	Only for 3

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Pointers	National Flag	partition using low, mid, high pointers	distinct values", "0s 1s 2s", "group by three categories"	an array with exactly 3 distinct values	end of 0s, high tracks start of 2s, mid scans; swap accordingly	pass, O(1) space	categories generaliz is harder

Runner Technique

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Two Pointers	Runner Technique	Advance one pointer by k steps first, then move both at the same speed	"kth from end", "N steps ahead"	When you need a fixed offset between two pointers	Move fast pointer k steps ahead; then advance both until fast reaches end	Finds kth-from-end in one pass	k must be ≤ length	Varies k dynamically

Meet in the Middle

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	V
Two Pointers	Meet in the Middle	Split problem into two halves, solve each independently, combine results	"n up to 40 elements", "subset sum with large n"	When $n \leq 40$ and brute force is 2^n — split	Generate all subsets of each half; sort one half, binary	Reduces exponential to manageable	Complex to implement; needs careful combination	V h ir o

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	V
				makes it $2^{(n/2)}$	search from other			

Bidirectional Scan

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Two Pointers	Bidirectional Scan	Compute answers from both left and right independently, then combine	"trap rain water", "product except self"	When the answer at position i depends on both left and right context	Build a left[] array and a right[] array in two passes; combine	Clean separation of concerns	Uses $O(n)$ extra space (can optimize)

Sliding Window

A sliding window is a contiguous subarray/substring that moves through the input. Think of it as a frame that slides across the data. The key intuition: instead of recomputing the window from scratch each time, you *add* one element on the right and *remove* one element on the left.

Core keywords to recognize sliding window: substring, subarray, contiguous, at most K , exactly K , minimum window, longest, shortest.

Fixed Window

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Sliding Window	Fixed Window	Window size is constant; slide it one step at a time	"window of size k", "every k elements", "k consecutive"	Fixed-size subarray problems	Add right element, remove leftmost element; maintain window aggregate	Simple, O(n)	Not useful when window size varies	Running sum, product, hash

Variable Window (Two Pointer Variant)

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Sliding Window	Variable Window	Window shrinks or expands based on a validity condition	"longest/shortest subarray with property", "at most K distinct"	Optimal-length subarray satisfying a constraint	Expand right pointer; when constraint violated, shrink from left	Handles complex constraints	Condition for shrinking must be precisely defined

Expand-Contract

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Va
Sliding Window	Expand-Contract	Grow the window greedily; contract	Any variable window problem	Maximizing a window under a constraint	Expand until invalid; contract	Very general technique	Shrinking logic depends heavily	Ini w loc sh

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Va
		only when necessary			until valid again; track max valid window seen		on problem	

Monotonic Window

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Sliding Window	Monotonic Window	Use a deque inside the window to track min/max in O(1)	"sliding window maximum/minimum"	When you need max or min of every window of size k	Maintain a deque in decreasing order for max (increasing for min); remove out-of-window elements from front	O(n) overall	Complex deque managem

Frequency Window

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Va
Sliding Window	Frequency Window	Maintain a character/element frequency map inside the window	"anagram in string", "permutation in string"	When window validity depends on	Keep a frequency map; compare with	O(n) time	Managing "valid count" requires care	Fix var wri

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Value
				element counts	target frequency map; slide and update			

Distinct Count Window

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Value
Sliding Window	Distinct Count Window	Track number of distinct elements in window	"at most k distinct characters/elements"	Longest/shortest window with ≤ k distinct values	Add element to map on expand; remove from map (and distinct count) on shrink when count hits 0	Handles distinct-count constraints elegantly		More robust to dynamic changes

Prefix-Assisted Window

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Value
Sliding Window	Prefix-Assisted Window	Combine prefix sums with a hash map to find subarrays	"subarray with sum exactly k", "zero-sum subarray"	When window shrinking isn't straightforward (negative)	Store prefix sum in map; if prefixSum	Works with negative numbers unlike	Doesn't give the actual subarray easily	Very efficient for negative numbers

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	V
		with a target sum		numbers present)	- k exists in map, found a valid subarray	pure sliding window		

Prefix Structures

A prefix structure precomputes cumulative information so that any range query [l, r] can be answered in O(1) by combining prefix[r] and prefix[l-1].

Insight: `rangeQuery(l, r) = prefix[r] - prefix[l-1]`

Prefix Sum

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants	C
Prefix Sum	Prefix Sum	Build prefix[i] = sum of arr[0..i]; range sum = prefix[r] - prefix[l-1]	"range sum", "subarray sum equals k", "number of subarrays with sum"	Static array with many range sum queries	One build pass O(n); each query O(1)	Extremely fast for multiple queries	Static — only updates require rebuild	With hash map for count, 2D prefix	F C S S k F I

Prefix XOR

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Prefix XOR	Prefix XOR	prefix[i] = XOR of arr[0..i];	"XOR of subarray",	Range XOR queries,	Same as prefix sum but	XOR is its own inverse,	Less intuitive	With hash map

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
		range XOR = prefix[r] XOR prefix[l-1]	"XOR queries"	finding subarrays with XOR equal to target	XOR is both the "add" and "undo" operation	making this elegant	than sum	to count subarrays

Prefix Product

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants	Complexity
Prefix Product	Prefix Product	prefix[i] = product of arr[0..i]; range product = prefix[r] / prefix[l-1]	"product of subarray", "product except self"	Range product queries when no zeros present	Division gives range product; handle zeros as special cases	O(1) query	Division by zero is a landmine	Left + right product arrays	Pro of A Exc Sel

Difference Array

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Prefix Difference Array	Difference Array	diff[l] += val, diff[r+1] -= val; prefix sum of diff gives the result	"range update", "add value to range", "flights/bookings"	Multiple range-update operations, single final read	Apply all updates to diff array; compute prefix sum once at end	O(1) per range update	Not for online queries needing intermediate state

2D Prefix Sum

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants	Con Prol Typ
Prefix	2D Prefix Sum	prefix[i][j] = sum of rectangle from (0,0) to (i,j); use inclusion-exclusion for submatrix	"sum of submatrix", "2D range query"	Grid problems with multiple rectangular sum queries	prefix[r2][c2] - prefix[r1-1][c2] - prefix[r2][c1-1] + prefix[r1-1][c1-1]	O(1) query after O(n²) build	Only for static grids	With hash for count	Sub Sun Que Max Squ

Hashing Patterns

Hash maps and sets reduce lookup from O(n) to O(1). They're the most versatile tool in interview problem-solving.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pro
Hashing	Frequency Map	Count occurrences of each element	"frequency", "count", "most common", "appears k times"	Any counting/frequency problem	Map element → count; query in O(1)	Uni
Hashing	Lookup Table	Store previously seen values for O(1) lookup	"two sum", "find complement", "check if seen before"	Searching for a "partner" element	Store element in set/map as you iterate; check if complement exists	Tui intc
Hashing	Index Mapping	Map element → its first/last index	"first repeated element", "longest subarray between two same elements"	Track position of elements	Map value → index; if seen again, compute distance	O(1
Hashing	Hash Set Presence	Use a set to check membership in O(1)	"contains duplicate", "missing number", "union/intersection"	Pure membership testing without counts	Insert elements into set; query set.contains()	Fas me tes
Hashing	Rolling Hash	Compute hash of a window incrementally as it slides	"find repeated substring", "string matching"	Efficient string comparison within sliding window	Hash = (hash * base + char - oldChar * power) % mod	O(1 sea
Hashing	Coordinate Compression	Map large sparse values to small dense indices	"values up to 10^9 but $n \leq 10^5$ "	When values are large but count is small; needed for segment tree indexing	Sort unique values; map each to its rank	Ma larg pro tra

SECTION 2: SEARCH + SORTING PATTERNS

Binary Search Family

Binary Search is valid whenever you can answer "is the answer in the left half or the right half?" — i.e., whenever there is **monotonicity**: the condition transitions from False to True (or True to False) at exactly one point.

The secret: Binary search isn't just for sorted arrays. It's for any **monotonic predicate** over a search space.

Classic Binary Search

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants	Complexity
Binary Search	Classic	Find exact target in sorted array	"sorted array", "find index of target"	Target element known, array sorted	lo=0, hi=n-1; mid=(lo+hi)/2; move lo or hi based on comparison	O(log n)	Off-by-one errors are common; use inclusive bounds carefully	Iterative vs recursive	1

Lower Bound / Upper Bound

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants	Complexity
Binary Search	Lower Bound	Find first position where element $\geq$ target	"first occurrence", "leftmost position", "at least"	Finding insertion point or first valid position	When arr[mid] $\geq$ target, move hi = mid; else	Handles duplicates correctly	Must clearly define $\geq$ vs $>$ predicate	Upper bound ($>$), exact match	1

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
					lo = mid+1			
Binary Search	Upper Bound	Find last position where element $\leq$ target	"last occurrence", "rightmost position"	Finding last valid position in sorted array	When arr[mid] $\leq$ target, lo = mid; else hi = mid-1	Pair with lower bound for count	Slightly different loop invariant	Combine with lower bound

Search on Answer Space

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Binary Search	Search Answer	Binary search on the answer value, not on an array index	"minimum/maximum possible value", "at least", "minimize the maximum", "capacity to ship"	When you need to find optimal value and a checker function is easy to write	Define: can we achieve answer = mid? If yes, try better; if no, try other direction	Converts optimization to decision	Must define monotonic checker carefully

Predicate Search

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Binary Search	Predicate Search	Binary search on a boolean	Any monotonic	Whenever the answer	Write f(mid); if f(mid)=True,	Generalizes all binary search	Monotonicity must be

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
		function $f(x)$ that transitions $F \rightarrow T$ or $T \rightarrow F$ exactly once	Yes/No question	space has a clear monotonic property	try left; else try right; answer is boundary		proven or intuited

Rotated Search

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Binary Search	Rotated Search	Binary search on array rotated at some pivot	"rotated sorted array", "originally sorted but shifted"	Sorted array with a single rotation point	At each mid, one half is always sorted — determine which half and decide where target lies	$O(\log n)$ on rotated arrays	Two rotation points? Use modified approach	With duplicates (worst case) $O(n)$

Peak Search

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variants
Binary Search	Peak Search	Find a local maximum where neighbors are smaller	"find peak element", "mountain array"	Any unimodal function; local peaks in arrays	If $arr[mid] < arr[mid+1]$, peak is to the right; else to the left	$O(\log n)$	Multiple peaks require careful handling	2D peak finding

Ternary Search

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Binary Search	Ternary Search	Find maximum/minimum of a unimodal continuous function	"minimize $f(x)$ for continuous x ", "maximum of unimodal"	Continuous unimodal functions where binary search doesn't directly apply	Divide range into thirds; compare $f(m1)$ and $f(m2)$; eliminate one third	Works on continuous domains	Slower than binary search by a constant factor; binary search on integers is usually enough

Sorting Based Patterns

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Sorting	Sort + Scan	Sort, then make a single linear pass	"does this array have X property after sorting"	Many array problems become trivial after sorting	Sort; observe structure; answer in one pass	$O(n \log n)$ total	Destroys original order
Sorting	Sort + Two Pointer	Sort first, then apply two pointer technique	"pairs/triplets in unsorted array"	Finding pairs/triplets after sorting makes two-pointer valid	Sort; apply opposite-ends or same-direction	$O(n \log n + n)$	Requires sorting
Sorting	Sort + Greedy	Sort by a key that makes greedy choices obvious	"interval scheduling", "assigning tasks"	When sorting creates a natural greedy order	Sort by start time, end time, or deadline; apply greedy rule	Greedy choices become locally obvious	Requires proving greedy validity
Sorting	Event Sorting	Create (time, type) events; sort; process in order	"meetings", "intervals", "overlapping events", "room allocation"	Sweep line problems; counting overlapping intervals	Convert intervals to start/end events; sort; use counter	Elegant for overlap counting	Two event at same time need tie-breaking
Sorting	Custom Comparator	Define a comparison function tailored to the problem	"custom order", "largest number from digits", "reorder by condition"	When natural sort order isn't what you need	Override comparator; sort; result is now in required order	Handles complex ordering	Custom comparators have subtle bugs



SECTION 3: INTERVAL PATTERNS

Intervals represent ranges [start, end]. The critical insight is that sorting by start time transforms a chaotic overlap problem into a left-to-right sweep.

Recognition keywords: start/end, range, meeting, schedule, overlap, booking, timeline, reservation

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Intervals	Merge Intervals	Combine overlapping intervals into one	"merge overlapping", "collapse ranges"	When you need non-overlapping representation	Sort by start; if current start $\leq$ prev end, merge; else add as new	$O(n \log n)$	Careful touch intervals (end : start)
Intervals	Overlap Detection	Detect if any two intervals overlap	"any conflict", "can all attend"	Checking feasibility of non-overlapping attendance	Sort by start; if next start < current end, overlap exists	$O(n \log n)$	Only detect doesn't resolve
Intervals	Interval Scheduling	Select maximum non-overlapping intervals	"maximum events", "maximum meetings you can attend"	Optimization: fit as many non-conflicting intervals	Sort by END time (greedy); greedily pick earliest-ending compatible interval	Classic greedy; provably optimal	Must end, start
Intervals	Meeting Rooms (Min Rooms)	Find minimum rooms needed for all intervals	"minimum rooms", "minimum resources", "maximum simultaneous"	Resource allocation problems	Use event sorting (+1 start, -1 end); track running maximum of concurrent count	$O(n \log n)$	Tie-break same matter
Intervals	Line Sweep	Process events at their x-coordinates left to right	"area of union of rectangles", "falling squares"	Geometric problems, area computation, coverage	Sort events; maintain active set; compute answer incrementally	Very general	Complex
Intervals	Difference Sweep	Apply difference	"range update then query",	When all updates are	diff[l]++, diff[r+1]--;	$O(n + m)$ for m	Cannot query

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
		array technique on interval ranges	"paint the fence"	range additions and only final state matters	prefix sum gives final coverage	updates	intermediate states

SECTION 4: LINKED LIST PATTERNS

Linked lists force you to think in terms of pointer manipulation rather than index arithmetic. The key insight: you can only move forward, so many tricks involve remembering "previous" pointers or using multiple pointers simultaneously.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
Linked List	Reverse List	Reverse all pointers in a singly linked list	"reverse a linked list"	Full reversal	Keep prev=null, curr=head; at each step: next=curr.next, curr.next=prev, prev=curr, curr=next	O(n) time, O(1) space
Linked List	Partial Reverse	Reverse only positions l to r	"reverse between positions", "reverse k segment"	Subrange reversal	Find node before l; reverse l to r; reconnect	O(n)
Linked List	K-Group Reverse	Reverse every k nodes as a group	"reverse k nodes at a time"	Groupwise reversal	Check if k nodes remain; reverse them; recurse or iterate on rest	Clean with recursion
Linked List	Dummy Node	Add a sentinel dummy node before head to simplify edge cases	Any problem where head might change or be deleted	Whenever head manipulation is needed	Create dummy.next = head; work with dummy; return dummy.next	Eliminates null-check special cases for head
Linked List	Fast-Slow Pointer	Slow moves 1 step, fast moves 2 — finds middle and cycles	"middle node", "cycle", "palindrome list"	Middle detection, cycle detection	Move until fast reaches end (or null) — slow is at middle	O(n), O(1)
Linked List	Cycle Detection	Detect and locate cycle start	"detect cycle", "find	Floyd's algorithm use case	After meeting point found: reset one	O(n), O(1)

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
			where cycle begins"		pointer to head; advance both 1 step at a time; meeting point is cycle start	
Linked List	Merge Two Lists	Merge two sorted linked lists into one sorted list	"merge sorted lists"	Combining sorted lists	Use dummy; compare heads of both; attach smaller; advance that list	$O(n+m)$
Linked List	Pointer Rewire	Rearrange node connections without creating/deleting nodes	"reorder list", "odd-even list", "weave lists"	Structural rearrangement in-place	Find middle; reverse second half; interleave	$O(n), O(1)$
Linked List	Intersection	Find node where two lists merge	"intersection point", "Y-shaped list"	Finding common suffix	Both pointers traverse both lists; length difference is eliminated naturally	Elegant $O(n+m)$ solution

SECTION 5: STACK PATTERNS

A stack is LIFO — Last In, First Out. The key power of a stack is its ability to **remember the most recent unprocessed element** as you scan left to right. Monotonic stacks extend this by maintaining a sorted invariant.

Recognition keywords: nearest, next greater, previous smaller, span, histogram, remove invalid, undo/backtrack-like behavior

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Stack	Balanced Parentheses	Match opening and closing brackets	"valid parentheses", "balanced", "matching brackets"	Any bracket/pair matching	Push opening; on closing, check if top matches; pop if yes	O(n), O(n)	Multiple bracket types need careful matching
Stack	Monotonic Increasing Stack	Stack where elements increase from bottom to top	"next smaller", "previous greater", "remove larger elements"	Finding nearest smaller to the right/left	Pop when current element is smaller than top; popped elements found their "next smaller"	O(n) total	Direction matters (left vs right scan)
Stack	Monotonic Decreasing Stack	Stack where elements decrease from bottom to top	"next greater", "previous smaller", "span"	Finding nearest greater to the right/left	Pop when current element is greater than top; popped elements found their "next greater"	O(n) total	"Next greater" or "previous greater" — scan direction changes
Stack	Next Greater Element	For each element, find the first greater element to its right	"next warmer day", "next larger element"	Classic monotonic stack problem	Scan left to right; maintain decreasing stack; when larger found, it's the answer	O(n)	Circular arrays need two-pass or modulo indexing

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					for all popped		
Stack	Histogram / Rectangle	Find largest rectangle in histogram	"largest rectangle in histogram", "maximal rectangle in matrix"	Maximum area under bars	For each bar, find how far left and right it extends as the shortest bar	O(n) with stack	Handling bars of equal height
Stack	Expression Parsing	Evaluate arithmetic expressions using a stack for values and operators	"evaluate expression", "basic calculator", "reverse polish notation"	Expression evaluation	Two-stack approach: one for numbers, one for operators; apply precedence rules	Handles precedence correctly	Parsing is verbose
Stack	Stack Simulation	Simulate a process that has undo/revisit behavior	"browser history", "undo redo", "directory path"	Any sequential process with backtracking	Use stack to store states; pop on "undo" or "go back"	Natural fit for recursive-like problems	Stack overflow if not bounded

SECTION 6: QUEUE PATTERNS

A queue is FIFO — First In, First Out. Its natural application is **level-by-level** processing. When BFS explores a graph, a queue guarantees that all nodes at distance d are processed before any node at distance $d+1$.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
Queue	BFS Queue	Standard BFS using a queue to process level by level	"shortest path in unweighted graph", "levels in tree", "minimum steps"	Unweighted shortest path, level-order traversal	Enqueue start; while queue not empty: dequeue, process, enqueue neighbors if not visited	Guaranteed shortest in unweighted graphs
Queue	Multi-Source BFS	Start BFS from multiple sources simultaneously	"nearest X to each cell", "rotting oranges", "walls and gates"	When multiple sources spread simultaneously	Enqueue all sources at once at level 0; run BFS normally	Handles simultaneous propagation naturally
Queue	Monotonic Queue (Deque)	Deque maintaining a monotonic order for sliding window extremes	"sliding window maximum/minimum"	When you need max/min of every window of size k	Maintain deque with decreasing order for max; remove front when out of window; remove back when smaller	O(n) for window
Queue	Priority Queue (Heap)	Queue ordered by priority rather than insertion time	"k-th largest", "always process smallest/largest next"	Greedy problems where next choice depends on current minimum or maximum	Offer/poll by priority; Java: PriorityQueue (min-heap by default)	O(log n) for operations
Queue	Scheduling Queue	Queue where tasks have arrival time	"CPU scheduling", "task ordering", "process jobs"	Simulating real-world scheduling	Sort by arrival; use priority queue for available	Combining sorting with heap

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
		and processing time			tasks; pick highest-priority available	

SECTION 7: HEAP PATTERNS

A heap is a partially sorted tree that gives $O(\log n)$ insert and $O(1)$ peek at the min/max. The critical use case: **you always need the best (min/max) of a dynamically changing set.**

Recognition keywords: top k, largest, smallest, stream, median, dynamically changing
best candidate

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Heap	Top K Elements	Maintain the k largest (or smallest) elements seen so far	"top k frequent", "k largest", "k closest"	Any "best k" problem	Use a min-heap of size k; add element; if heap size > k, poll (removes smallest); remaining k are the largest	$O(n \log k)$ — better than sorting when $k \ll n$	$k \log n$
Heap	Running Median	Find median of a data stream after each insertion	"median of stream", "sliding median"	Dynamic median under insertions	Two heaps: max-heap for lower half, min-heap for upper half; keep sizes balanced	$O(\log n)$ per insertion, $O(1)$ median	Complexity
Heap	Two Heap	Maintain two heaps representing two halves of a distribution	Any problem needing simultaneous access to max of left half and min of right half	Balancing or splitting data	Max-heap + min-heap; top of each is boundary; rebalance on every insert	Very fast queries	No imbalance
Heap	Merge K Lists	Merge k sorted sequences using a heap	"merge k sorted lists", "k sorted arrays", "external merge sort"	Merging multiple sorted sources	Insert first element of each list into min-heap; poll gives next smallest;	$O(n \log k)$ where n is total elements	Recall list allocation

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					push its successor		
Heap	Greedy Heap	At each step, pick the greedy best using a heap	"always do the task with the highest/lowest X first"	Greedy algorithms needing dynamic minimum/maximum	Build heap; pop best option; update state; push new option	Combines greedy correctness with efficient data structure	Mutates greedy operation
Heap	Stream Processing	Process elements one-by-one, maintaining heap-based invariant	"k-th element in a stream", "online algorithm"	When data arrives one at a time	Maintain heap of fixed size; each new element potentially displaces heap min/max	Online algorithm — no need to see all data	Not quite

SECTION 8: RECURSION + BACKTRACKING

Recursion is the art of solving a problem by breaking it into smaller identical problems.

Backtracking is recursion with the additional idea of **undoing choices** when they lead to dead ends.

Mental model: Think of a decision tree. At each node, you make a choice. If the path fails, you backtrack to the parent and try another branch.

Recognition keywords: generate all, all possibilities, all paths, decision tree, explore state space, count arrangements, enumerate

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	P
Backtracking	Include- Exclude	At each element, decide: include it in current set or exclude it	"subsets", "power set", "sum combinations"	Subset enumeration	At each index, branch: add current element and recurse, then remove and recurse without it	Generating 2^n
Backtracking	Choose- Explore- Unchoose	Make a choice, explore consequences, undo the choice	Any backtracking problem	Universal backtracking template	choose(option); recurse(); unchoose(option) — this is the core template	Complex systems: e.g. N-Queens
Backtracking	Permutation	Generate all orderings of a set of elements	"all arrangements", "all orderings", "permutations"	Permutation enumeration	Use a visited/used boolean array; at each position, try each unused element	Generating n!
Backtracking	Combination	Generate all size-k subsets	"combinations", "choose k from n"	Fixed-size subset enumeration	Start from current index; choose elements one by one; recurse from next index onward	Generating combinations (C(n, k))
Backtracking	N-Queens	Place N queens on N×N board such that none attack each other	"N queens", "safe placement", "constraint placement"	Constraint satisfaction with position conflicts	For each row, try each column; check row/column/diagonal conflicts; backtrack on conflict	Complex constraint satisfaction
Backtracking	Sudoku / Constraint Search	Fill cells under constraints; backtrack when stuck	"fill grid", "sudoku", "constraint satisfaction"	Grid constraint satisfaction	Find empty cell; try values 1-9 (or whatever range); check validity; recurse; backtrack if stuck	Solving constraint satisfaction

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	P
Backtracking	Branch Pruning	Cut off recursive branches early when they can't lead to a valid answer	Any backtracking with a clear "too large", "impossible" condition	Improves efficiency of backtracking	Add condition at start of recursion: if current path is already invalid, return immediately	C fr e: tc n

SECTION 9: TREE PATTERNS

Trees are the recursive data structure — every subtree is itself a tree. This means almost every tree problem has a natural recursive solution. The traversal order (pre, in, post, level) determines when you process a node relative to its children.

Key intuition for traversal choice:

- **Preorder (Root → Left → Right):** Use when you need to process a node *before* its children. Tree cloning, serialization.
- **Inorder (Left → Root → Right):** Gives sorted order in BST. Use for sorted processing.
- **Postorder (Left → Right → Root):** Use when you need children's results *before* deciding about parent. Tree deletion, height/balance computation.
- **Level Order (BFS):** Use when you need to process nodes layer by layer. Level-based problems.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Tree	Preorder Traversal	Process node, then left subtree, then right subtree	"path from root", "clone tree", "serialize"	When current node must be processed before children	Visit node → recurse left → recurse right	Natural for top-down problems	Root value is processed before knowing subtree results
Tree	Inorder Traversal	Process left subtree, then node, then right subtree	"sorted order in BST", "in-order sequence"	BST problems; producing sorted output	Recurse left → visit node → recurse right	Gives sorted BST elements	Doesn't expose root-first info
Tree	Postorder Traversal	Process left subtree, right subtree, then node	"height", "diameter", "is balanced", "tree deletion"	When subtree results needed before parent decision	Recurse left → recurse right → visit node	Natural for bottom-up aggregation	Must wait for both children before acting
Tree	Level Order	Process all nodes at each level before moving deeper	"level-by-level", "zigzag", "right side view", "cousins"	Any level-based problem	BFS with queue; at each level, process all nodes in current queue	Enables per-level processing	More complex than DFS
Tree	DFS Tree DP	Each node returns a value computed from its children's values	"tree DP", "optimal path", "aggregate over tree"	Computing global answer from local subtree results	Postorder traversal; each node gets left and right subtree results; computes	Very powerful paradigm	State design can be tricky

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					its contribution		
Tree	Diameter	Find longest path between any two nodes (need not pass through root)	"diameter", "longest path between two nodes"	Computing tree diameter	At each node: diameter through node = leftHeight + rightHeight; update global max	Clean postorder solution	Path doesn't always pass through root — need global variable
Tree	LCA	Find the lowest common ancestor of two nodes	"LCA", "common ancestor", "meet at", "path between two nodes"	Ancestor queries	Postorder: if both nodes found in different subtrees, current node is LCA	O(n) per query, O(n) with Euler tour + RMQ	Naive is O(n) per query
Tree	Path Sum	Find if a root-to-leaf path sums to a target	"path sum", "target from root to leaf"	Root-to-leaf path problems	Subtract current node's value as you recurse; base case: leaf with remaining = 0	Simple recursive	All paths specific paths: different variants
Tree	Serialization	Convert tree to string and back	"serialize", "deserialize", "encode", "decode"	Transmitting or storing a tree	Preorder + null markers; or level order BFS with null markers	Unique encoding with null markers	BFS and DFS give different formats

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Tree	Morris Traversal	Inorder traversal with O(1) space using threaded binary tree	"inorder in O(1) space"	Space-constrained inorder traversal	Temporarily make right child point back to parent; undo after visiting	O(1) space	Complex modifies tree temporarily

SECTION 10: BST-SPECIFIC TREE PATTERNS

BST has a defining property: every node's left subtree contains only smaller values, right subtree only larger values. This structural property enables $O(\log n)$ operations (in balanced BSTs) and makes inorder traversal produce sorted output.

How BST problems differ from general tree problems: In general trees, you often need to explore all subtrees. In BST, the ordering property tells you *which subtree* to explore, potentially halving the search space at each step.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
BST	Validate BST	Verify BST property holds for entire tree	"is valid BST", "check BST"	Validation problems	Pass min and max bounds down; at each node, value must be in (min, max)	$O(n)$, one pass	Comparing node values directly is wrong — use bounds
BST	Kth Smallest / Largest	Find kth element in BST order	"kth smallest", "kth largest in BST"	Ranked element queries	Inorder traversal (sorted order); count until k-th reached	$O(n)$ worst, $O(k)$ if augmented with subtree sizes	$O(n)$ without augmentation
BST	Inorder Successor	Find next node in inorder sequence	"inorder successor", "next node in BST"	Ordered iteration	Case 1: if right child exists, go right then leftmost; Case 2: go up until you come from a left child	$O(h)$	Need parent pointers for Case 2
BST	Range Search	Find all nodes within a value range [lo, hi]	"range query in BST", "count nodes in range"	Range counting/listing	If node < lo, go right; if node > hi, go left; else add and	$O(\log n + k)$ where k is output size	Only efficient in balanced BST

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					explore both		
BST	Convert Sorted Array to BST	Build balanced BST from sorted array	"convert sorted array/list to height-balanced BST"	Constructing balanced BST	Mid element = root; recurse on left half and right half	Always produces balanced result	Only for sorted input
BST	LCA in BST	Exploits BST property to find LCA efficiently	"LCA in BST"	LCA with BST property	If both nodes < root, go left; if both > root, go right; else root is LCA	$O(h)$ — faster than general tree LCA	Only for BST
BST	Vertical Order / Top / Bottom View	Process tree column by column	"vertical order", "top view", "bottom view"	Column-based tree problems	BFS with column index tracking; use TreeMap for ordered columns	Systematic	Column indexing needs careful initialization
BST	Boundary Traversal	Traverse left boundary + leaves + right boundary	"boundary of binary tree"	Tree boundary problems	Left boundary top-down; leaves left-to-right; right	Three separate passes are clean	Avoiding duplicates at corners

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					boundary bottom-up		
BST	Zigzag Level Order	Level order with direction alternating each level	"zigzag", "spiral order traversal"	Level-order with alternating direction	BFS with level direction flag; use deque or reverse alternate levels	Clean with deque	Direction tracking must be consistent

SECTION 11: GRAPH PATTERNS

Graphs generalize trees by allowing arbitrary connections and cycles. The two fundamental traversals (BFS and DFS) power most graph algorithms; everything else (shortest path, cycle detection, spanning tree) builds on top of them.

Traversal

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Graph	BFS	Explore all neighbors before going deeper	"shortest path", "levels", "minimum steps"	Unweighted shortest path, level-by-level exploration	Queue + visited set; enqueue start; process level by level	Guaranteed shortest path in unweighted graphs	Memory intensive for wide graphs
Graph	DFS	Explore one path as deep as possible	"connected components", "cycle"	Connectivity, cycle detection,	Stack/recursion + visited set; explore path	Memory-efficient for deep graphs;	Not shortest path

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
		before backtracking	detection", "all paths"	topological sort	fully before backtracking	simple recursive	

Connectivity

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Graph	Union Find (DSU)	Group elements into disjoint sets; merge groups	"connected components", "dynamic connectivity", "detect cycle in undirected"	Dynamic connectivity, grouping, cycle detection	parent[] array; find(x) returns root; union(x,y) merges roots; path compression + rank	Nearly O(1) per operation with optimization	Not for directed graphs
Graph	Cycle Detection (Directed)	Detect cycle in directed graph using DFS coloring	"deadlock", "circular dependency"	Directed graphs	3-color DFS: white (unvisited), gray (in stack), black (done); gray→gray = cycle	Reliable	Undirected cycle needs parent-tracking variant
Graph	Bipartite Check	Determine if graph can be 2-colored with no same-color neighbors	"bipartite", "two teams", "odd cycle check"	Conflict graphs, assignment problems	BFS/DFS with 2-coloring; if neighbor has same color → not bipartite	Detects odd cycles	Not for weighted or directed graphs

Shortest Path

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Variations
Graph	Dijkstra	Single-source shortest path in weighted graph with non-negative edges	"shortest path", "minimum cost", "weighted edges"	Non-negative weighted graphs	Min-heap (priority queue) of (dist, node); relax neighbors; skip if already finalized	$O((V+E) \log V)$	Fails with negative edges	Bidirectional Dijkstra
Graph	Bellman-Ford	Single-source shortest path; handles negative weights	"negative weights", "detect negative cycle"	Graphs with negative edges	Relax all edges $V-1$ times; V -th iteration still relaxes $\rightarrow$ negative cycle exists	Handles negative edges	$O(VE)$ — slow	SPFA / optimizations
Graph	Floyd-Warshall	All-pairs shortest path	"shortest path between all pairs", "reachability matrix"	Small dense graphs needing all-pairs distances	$dp[i][j] = \min \text{ over } k$	$O(V^3)$; easy to implement	$O(V^3)$ — only for small graphs	Transitive closure
Graph	0-1 BFS	Shortest path in graph with edge weights 0 or 1	"move with cost 0 or 1", "two types of edges"	Binary-weight edge graphs	Deque: cost-0 edges go to front, cost-1 edges go to back	$O(V+E)$ — faster than Dijkstra for 0-1	Only for 0 and 1 weights	Generalization to integer weights using bucket

Ordering

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Notes
Graph	Topological Sort (Kahn's BFS)	Order nodes of DAG such that all edges go left-to-right	"prerequisites", "dependency order", "build order"	DAG ordering	Compute in-degrees; enqueue all zero in-degree nodes; process and decrement; repeat	Detects cycles (incomplete ordering = cycle)	Directed acyclic graphs only	For DAGs
Graph	Topological Sort (DFS)	Post-order DFS; reverse gives topological order	Same as Kahn	DAG ordering	DFS with finish-time tracking; reverse finish order is topological order	Same result as Kahn	Slightly more complex	For DAGs

Special Graph Algorithms

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Graph	MST Kruskal	Minimum spanning tree by sorting edges and using DSU	"minimum cost to connect all", "MST"	Sparse graphs; edge list available	Sort edges by weight; add edge if it doesn't form cycle (DSU); stop	$O(E \log E)$; simple	Requires sorted edges

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					after V-1 edges		
Graph	MST Prim	Minimum spanning tree by growing from a start node	Same as Kruskal	Dense graphs; adjacency list available	Use priority queue; greedily pick minimum-weight edge crossing the cut	$O((V+E) \log V)$ with heap	More complex than Kruskal
Graph	Bridges & Articulation Points	Find edges/nodes whose removal disconnects the graph	"critical connections", "bridges", "cut vertices"	Network reliability problems	Tarjan's algorithm with DFS timestamps and low[] values	$O(V+E)$	Complex to implement
Graph	SCC (Kosaraju / Tarjan)	Find strongly connected components	"strongly connected", "condensation graph"	Directed graphs needing component structure	Kosaraju: 2-pass DFS (original + transpose); Tarjan: single DFS with low-link values	$O(V+E)$	Complex
Graph	Flood Fill / Island Pattern	Count or explore connected regions in a grid	"number of islands", "count regions", "area of island"	Grid connectivity problems	DFS or BFS from each unvisited land cell; mark visited	Simple $O(n*m)$	Multiple directions needed (or 8) directions

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Graph	State-Space Graph	Model problem states as nodes, transitions as edges	"minimum steps", "word ladder", "unlock combination", "minimum moves to solve"	Problems where state = node, action = edge	Encode state as node; BFS for minimum steps to reach target state	Converts complex simulation to graph problem	State space can be huge
Graph	Multi-Source BFS	Start BFS from multiple sources simultaneously	"nearest 0", "rotting oranges", "walls and gates"	Simultaneous spreading from multiple origins	Add all sources to queue at time 0; BFS normally	Handles multiple origins in one pass	All sources must be known upfront

SECTION 12: GREEDY PATTERNS

A greedy algorithm makes the **locally optimal choice** at each step, hoping it leads to a global optimum. The art of greedy is not in the implementation (which is usually simple) but in **proving that local optimality implies global optimality**.

How to verify greedy validity: Use an exchange argument — assume an optimal solution differs from your greedy at some position; show you can swap to match the greedy choice without worsening the result.

When greedy fails: When future choices invalidate the current "locally best" choice. This is the signal to use DP instead.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Greedy	Activity Selection	Select maximum number of non-overlapping intervals	"maximum non-overlapping", "maximum events you can attend"	Interval scheduling maximization	Sort by end time; greedily pick earliest-ending activity that doesn't conflict	Provably optimal	Must sort by end time, not start time or length
Greedy	Jump Game	Determine if you can reach the end of an array	"jump game", "can you reach the end?"	Reachability in array with jump sizes	Maintain maximum reachable index; update as you scan; if current position > max reachable, stuck	$O(n)$, $O(1)$	Only for single target (can't reach end tracking multiple adjustments)
Greedy	Scheduling / Task Assignment	Assign tasks to minimize makespan or meet deadlines	"minimum time", "deadline scheduling", "least idle CPU time"	Task scheduling with constraints	Sort tasks; assign to earliest available slot; use heap for available resources	Typically $O(n \log n)$	Complex constraint invalidate greedy
Greedy	Huffman Encoding	Build optimal prefix-free encoding using a min-heap	"minimum total encoding length", "Huffman"	Variable-length encoding problems	Always merge the two least-frequent symbols; result is	Provably optimal	Not directly asked in many interviews; concept used in other contexts

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
					optimal prefix tree		
Greedy	Resource Allocation	Allocate limited resources optimally	"assign bikes", "match people to resources"	Bipartite matching-lite, assignment	Sort both sides; greedily assign closest or best match	Usually $O(n \log n)$	Complex preferences break simple greedy
Greedy	Greedy with Heap	Combine greedy choices with a heap for dynamic best selection	"reorganize string", "always pick most frequent"	When best candidate changes dynamically	Heap stores candidates; pop best; process; push updated candidates	Dynamic greedy selection	Implementation complexity

SECTION 13: DYNAMIC PROGRAMMING

DP is the technique of solving overlapping subproblems by storing their results. It applies when a problem has **optimal substructure** (optimal solution built from optimal sub-solutions) and **overlapping subproblems** (same subproblem solved multiple times).

The DP mindset: Define state → write transition → handle base cases → optimize space.

Recognition keywords: maximize, minimize, ways, count, optimal, choose best among many options, is it possible

Foundation

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
DP	Memoization (Top-Down)	Recursion + cache; compute only needed subproblems	Natural recursive structure; overlapping calls	When recursion is intuitive; subproblems overlap	Write recursive solution; add HashMap/array cache; return cached if available	Intuitive; only computes needed states	Stack overflow risk if deep recursion
DP	Tabulation (Bottom-Up)	Iterative; fill DP table from smallest subproblems	Same as memoization	When you know the order of subproblems	Define dp[]; fill base cases; iterate forward; each state depends only on earlier states	No recursion overhead; cache-friendly	Must determine correct filling order
DP	Space Optimization	Reduce DP table from $O(n^2)$ to $O(n)$ or $O(1)$	Same DP problem but with tight memory constraint	When current row depends only on previous row(s)	Use two arrays or variables instead of full table	Significant memory saving	Loses ability to record solution path

DP Families

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach
DP	0/1 Knapsack	Include or exclude each item; each item usable at most once	"select items with weight/value", "subset selection"	Item selection with capacity constraint	$dp[i][w] = \max$ value using first i items with capacity w ; for item i : include or exclude

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach
DP	Unbounded Knapsack	Items can be reused unlimited times	"coin change", "complete", "unlimited supply"	Unlimited item selection	$dp[w] = \min/\max$ over all items where $item.weight \leq w$; inner loop has no "exclude" step
DP	Subset Sum / Partition	Determine if a subset sums to a target	"partition into equal", "can you form sum S", "target subset"	Subset selection with sum constraint	$dp[j]$ = can we form sum j; for each item, update backwards
DP	LCS (Longest Common Subsequence)	Find longest sequence present in both strings (not necessarily contiguous)	"common subsequence", "edit distance flavour", "deletion operations"	String comparison problems	$dp[i][j]$: LCS of $s1[0..i-1]$ and $s2[0..j-1]$; if chars match, extend; else take max of skip either
DP	LIS (Longest Increasing Subsequence)	Find longest strictly increasing subsequence	"longest increasing", "chain of pairs", "Russian doll envelopes"	Increasing sequence problems	$dp[i]$ = LIS ending at index i; for each $j < i$: if $arr[j] < arr[i]$, $dp[i] = \max(dp[i], dp[j]+1)$
DP	String DP	Edit distance, pattern matching, wildcard	"edit distance", "wildcard match", "regex match"	String transformation/matching	$dp[i][j]$ encodes state of first i chars of s and j chars of t
DP	Grid DP	Move through a grid, accumulating cost/paths	"paths in grid", "minimum cost path", "unique paths"	Grid traversal optimization	$dp[i][j]$ = optimal result at cell (i,j); comes from $dp[i-1][j]$ or $dp[i][j-1]$

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach
DP	Interval DP	Solve problems on intervals by merging smaller intervals	"burst balloons", "matrix chain multiplication", "minimum cost to merge"	Problems where subproblems are sub-intervals [i,j]	$dp[i][j]$ = answer for range [i,j]; iterate by increasing length; try all split points k
DP	Tree DP	Compute DP values on each node from its children	"maximum independent set on tree", "house robber on tree"	DP on tree structure	Postorder traversal; each node gets aggregated result from children
DP	Digit DP	Count numbers in range [L, R] satisfying a digit-based condition	"count numbers with property X", "numbers without digit Y in range"	Counting numbers with digit constraints	State = (position, tight, sum/other_condition count valid completions at each position
DP	Bitmask DP	Use bitmask to represent subset of elements as DP state	"travelling salesman", "assign tasks to workers", "cover all cities"	Small n (≤ 20) with subset state	$dp[mask][node]$ = optimal result having visited set represented by mask
DP	State Machine DP	Model problem as states with transitions; DP over states	"buy/sell stock with cooldown", "finite automaton", "states with rules"	Problems with distinct modes/phases	Define states (e.g., hold, sold, cooldown) transition between states with cost
DP	Profile DP	Fill grid/array row by row using bitmask of	"tiling problems", "number of	Row-by-row grid filling with dependencies	$dp[mask]$ represents valid configurations of current row given previous row's mask

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach
		current row's state	ways to fill grid"		

SECTION 14: BIT MANIPULATION

Bits are the lowest level of abstraction in computation. Bit manipulation patterns exploit properties of binary representation for $O(1)$ or $O(\log n)$ solutions to problems that would otherwise require iteration.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Bits	XOR Trick	XOR of identical values cancels out (a XOR a = 0)	"find single number", "find missing", "find different element"	When pairs cancel and only one remains	XOR all elements; pairs cancel; result is the unpaired element	O(n), O(1)	Only works when pairs exist
Bits	Bit Mask	Use integer as boolean array to represent subset membership	"is element i in set?", "subsets"	Set representation with small universe	Bit i in mask represents element i; set: mask	$= (1 \ll i)$; check: mask & $(1 \ll i)$	O(1) operations O(1) space
Bits	Set / Unset / Toggle Bit	Modify individual bits	"set bit at position k", "clear bit", "flip bit"	Low-level bit manipulation	Set: n	$= (1 \ll k)$; Unset: $n \&= \sim (1 \ll k)$; Toggle: $n \wedge= (1 \ll k)$	O(1)
Bits	Subset Enumeration	Enumerate all subsets of a set using bit tricks	"iterate over all subsets of mask"	Bitmask DP, when all subsets needed	for (int sub = mask; sub > 0; sub = (sub-1) & mask) iterates all non-empty subsets	$O(3^n)$ total across all masks	Complex iteration pattern
Bits	Brian Kernighan (Count Bits)	$n \& (n-1)$ removes the lowest set bit	"count set bits", "is power of 2"	Counting 1-bits in a number	Repeat $n \&= (n-1)$ until $n=0$; count iterations	O(number of set bits)	Not useful for finding bit position

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Bits	Prefix XOR	cumXOR[i] = XOR of arr[0..i]; range XOR = cumXOR[r] XOR cumXOR[l-1]	"XOR of subarray", "find subarray with XOR=k"	XOR range queries	Same as prefix sum but with XOR as the operator	XOR is its own inverse	Less intuitive than sum
Bits	XOR Trie	Trie built on bit representation of numbers for XOR maximization	"maximum XOR of two numbers", "maximum XOR in subarray"	Finding maximum XOR pair efficiently	Insert numbers bit by bit into trie; for each query, greedily pick opposite bit	$O(32 \cdot n)$	Complex implementation

SECTION 15: TRIE PATTERNS

A Trie (prefix tree) stores strings character by character, enabling $O(L)$ operations where L is string length, regardless of how many strings are stored. Its killer advantage: **prefix sharing** — all strings with common prefix share the same path.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Trie	Prefix Search	Find all strings starting with a given prefix	"autocomplete", "starts with", "prefix matching"	Dictionary prefix queries	Insert strings into trie; follow path for prefix; DFS from endpoint gives all matches	$O(L)$ per query; beats hashing for prefix queries	$O(L)$ space
Trie	Word Dictionary with Wildcards	Search where some characters can be wildcards	"word search with dots", "regex-like matching"	Approximate string matching	Standard trie traversal; on wildcard, branch to all children	Handles wildcards naturally	Waste of space if many branches are not used
Trie	XOR Trie	Trie on binary representation of integers	"maximum XOR pair", "maximum XOR with prefix"	XOR optimization problems	Insert numbers bit by bit (MSB first); query by following opposite bits greedily	$O(32)$ per operation	Fixed size
Trie	Compressed Trie (Radix Tree)	Merge chains of single-child nodes	"space-efficient trie"	When strings share long common prefixes	Merge single-child chains into one edge	$O(n)$ space instead of $O(n \cdot L)$	Complex to implement

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Trie vs HashMap	When trie beats hash	Hash map gives $O(L)$ per operation too, but trie allows prefix enumeration	"list all words with prefix X"	Prefix enumeration, common prefix counting	Trie stores prefix structure explicitly; hash map doesn't	Prefix operations; longest common prefix	Higher space complexity

SECTION 16: RANGE QUERY DATA STRUCTURES

These structures answer repeated range queries efficiently. The choice depends on whether updates are needed and how frequent queries are.

Quick guide:

- Static array, range sum only → **Prefix Sum** ($O(1)$ query, $O(n)$ build)
- Static array, range min/max → **Sparse Table** ($O(1)$ query, $O(n \log n)$ build)
- Dynamic updates + range queries → **Segment Tree** or **Fenwick Tree** ($O(\log n)$ both)
- Range updates + range queries → **Lazy Segment Tree** ($O(\log n)$ both)

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Range DS	Segment Tree	Binary tree over array; each node stores aggregate of a range	"range sum/min/max with updates"	Dynamic arrays with frequent range queries	Build in $O(n)$; each internal node = merge of children; query/update in $O(\log n)$	Very flexible; handles any associative operation	$O(4n)$ space; complex implementation
Range DS	Lazy Propagation	Defer range update; only apply when actually needed	"range update + range query"	Segment tree with range updates	Store pending lazy value at each node; push down before querying children	$O(\log n)$ per range update	Complex manager
Range DS	Fenwick Tree (BIT)	Space-efficient structure for prefix sum with point updates	"prefix sum with updates", "inversions count"	Point updates + prefix sum queries	Binary Indexed Tree; update and query in $O(\log n)$; simpler than segment tree	$O(n)$ space; fast in practice	Only for prefix based operation range mir
Range DS	Sparse Table	Precompute answers for all ranges of power-of-2 length	"static range min/max with many queries", "RMQ"	Immutable arrays with heavy range min/max querying	Build $dp[i][j]$ = min/max of range $[i, i+2^j-1]$; query combines two overlapping blocks	$O(1)$ query	$O(n \log n)$ space; no updates
Range DS	Difference Array	$O(1)$ range updates; prefix sum at end gives final array	"apply range increments, read final array"	Multiple range-add updates, single read	$diff[1] += val$, $diff[r+1] -= val$; prefix sum gives result	$O(1)$ per update	Only one read; no intermediary queries

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
Range DS	Interval Tree	Augmented BST for interval overlap queries	"find all intervals overlapping with query", "stabbing query"	Interval storage with overlap queries	Each node stores max endpoint in subtree; prune search using max value	$O(\log n + k)$ per query	Complex implementation

SECTION 17: ADVANCED ALGORITHMIC PARADIGMS

Divide and Conquer

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons	Value
D&C	Divide and Conquer	Split problem into halves; solve each; merge	"count inversions", "merge sort", "closest pair"	Problems where cross-split combinations can be handled in merge step	Split at mid; solve left and right; merge with $O(n)$ or $O(n \log n)$ step	$O(n \log n)$ for many problems	Must define a meaningful merge	D8

String Matching Algorithms

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
String	KMP (Knuth-Morris-Pratt)	Pattern matching without backtracking using failure function	"find pattern in text", "pattern occurrences"	Linear-time substring search	Build prefix function (failure function) for pattern; use it to skip redundant comparisons	$O(n+m)$ — optimal
String	Z Algorithm	$z[i]$ = length of longest substring starting at i that matches a prefix of string	"pattern in text", "prefix matching"	Linear-time substring search; similar to KMP	Build z-array; concatenate pattern+"\$"+text; find $z[i]$ = pattern length	$O(n+m)$; simpler than KMP
String	Rabin-Karp	Rolling hash for pattern matching; allows multi-pattern matching	"repeated substring", "find duplicate substrings", "multiple patterns"	Multiple pattern search; duplicate detection	Compute hash of window; slide and update hash incrementally	Expected $O(n+k)$
String	Manacher's Algorithm	Find longest palindromic substring in $O(n)$	"longest palindrome", "palindrome count"	All palindromic substrings in linear time	Transform string (insert separators); compute $p[i]$ = radius of palindrome at center i ; expand using previous results	$O(n)$ — optimal
String	Suffix Array	Sorted array of all suffixes of a string	"longest common substring", "count distinct substrings"	Multiple complex string queries on one string	Build suffix array with $O(n \log n)$ or $O(n)$; compute LCP array	Powerful; $O(n \log n)$ for many operations

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
String	Aho-Corasick	Multi-pattern trie with failure links for simultaneous matching	"find all patterns in text simultaneously"	Multiple pattern search at once	Build trie of all patterns; add failure links (like KMP failure function on trie); single text scan	$O(n + \text{sum_pattern_len} + \text{output})$

Other Advanced Paradigms

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
Advanced	Matrix Exponentiation	Compute nth term of linear recurrence in $O(\log n)$ using matrix power	"nth Fibonacci in $\log n$ ", "count paths of length n in graph"	Linear recurrences with large n	Express recurrence as matrix multiplication; use fast power on matrix	$O(k^3 \log n)$ for k -state recurrence
Advanced	Binary Lifting	Precompute 2^k -th ancestors for $O(\log n)$ LCA and ancestor queries	"kth ancestor", "LCA in $\log n$ "	LCA queries on trees	For each node, store 2^k -th ancestor for $k=0..\log n$; LCA by lifting both nodes	$O(n \log n)$ build, $O(\log n)$ query
Advanced	Mo's Algorithm	Offline range queries processed in optimal	"range queries offline", "sqrt decomposition queries"	Many offline range queries where add/remove	Sort queries by block of left endpoint, then right endpoint;	$O((n+q)\sqrt{n})$

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
		order to minimize updates		from range is easy	move [l,r] boundaries	
Advanced	Coordinate Compression	Map large sparse values to compact indices	"values up to 10^9 but $n \leq 10^5$ "	When value magnitude is large but count is small	Collect all values; sort; map each to rank	Makes segment tree/BIT c values feasible
Advanced	Convex Hull	Find smallest convex polygon containing all points	"convex hull", "farthest points"	Computational geometry	Graham scan or Jarvis march; sort by angle; monotone chain	$O(n \log n)$

SECTION 18: STRING PATTERNS

Strings deserve their own section because the same string can be queried in many ways: as an array of characters, as a sequence (subsequence problems), as a window (substring problems), or by its prefix structure.

Key recognition: Is the problem about a **substring** (contiguous), **subsequence** (non-contiguous but ordered), or just **set of characters** (anagram/frequency)?

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
String	Anagram Pattern	Two strings are anagrams iff they have identical character frequencies	"anagram", "permutation of", "same characters"	Anagram checking/finding	Sort both strings, or compare frequency arrays	$O(L)$ with frequency array	Case sensitive Unicode characters
String	Palindrome (Expand Around Center)	Expand from center outward to find palindrome extent	"longest palindromic substring"	Finding palindromes in $O(n^2)$	For each center (n odd, n-1 even), expand while chars match	$O(n^2)$; simple to implement	$O(n)$ Manacher's algorithm for longer strings
String	Isomorphic / Pattern Mapping	Two strings are isomorphic if there's a consistent character-to-character mapping	"isomorphic strings", "word pattern", "follows pattern"	Structural pattern matching	Use two maps: one forward ($s \rightarrow t$), one backward ($t \rightarrow s$); ensure bijection	$O(L)$	Both directions must be checked (bijection)
String	Word Break	Determine if string can be split into dictionary words	"word break", "can you segment string"	String segmentation with dictionary	DP: $dp[i] = \text{true}$ if $s[0..i-1]$ can be segmented; for each i , try all splits	$O(n^2 * L)$	Can use trie for speed dictionary lookup
String	Edit Distance	Minimum operations (insert, delete, replace) to	"edit distance", "minimum operations"	String transformation problems	$dp[i][j] = \text{edit distance of } s1[0..i-1] \text{ to } s2[0..j-1]$;	$O(nm)$	Large strings space optimization

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros	Cons
		convert one string to another	"convert string"		transitions for match, insert, delete, replace		
String	Wildcard / Regex Matching	Match string against pattern with wildcards (* and ?)	"wildcard match", "regex", "pattern match with **"	Pattern matching with special characters	dp[i][j] = does s[0..i-1] match p[0..j-1]; handle * specially (matches any sequence)	O(nm) DP	* is the tricky c
String	Repeated Substring Pattern	Check if string is formed by repeating a shorter string	"repeated pattern", "basic period of string"	Periodicity problems	String s has a repeated pattern iff s is a substring of (s+s) excluding first and last character	O(n) with KMP	Non-tr why s+ trick w
String	Longest Common Prefix	Find prefix shared by all strings in a set	"common prefix", "shortest prefix"	Group prefix identification	Sort strings; compare first and last; their common prefix is the answer for all	O(n log n + L)	Sorting change order

SECTION 19: NUMBER THEORY + MATH PATTERNS

Math patterns let you replace brute-force loops with formulas, reducing $O(n)$ or $O(n^2)$ to $O(1)$ or $O(\log n)$.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
Math	GCD / LCM	GCD via Euclidean algorithm; LCM = $a*b/\text{GCD}(a,b)$	"common divisor", "common multiple", "reduce fraction"	Simplification, alignment problems	$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$; base case $\text{gcd}(a, 0) = a$	$O(\log \min(a, b))$
Math	Prime Sieve	Find all primes up to n in $O(n \log \log n)$	"primes up to n ", "count primes", "is prime"	When you need all primes in a range	Sieve of Eratosthenes: mark composites; primes are unmarked	$O(n \log \log n)$ very fast
Math	Modular Arithmetic	Compute $(a \text{ op } b) \bmod m$ without overflow	"mod 10^9+7 ", "result modulo", "last digit"	Any combinatorics or counting problem with large output	$(a+b)\%m = ((a\%m) + (b\%m))\%m$; same for multiplication; division needs modular inverse	Prevents overflow
Math	Fast Power (Binary Exponentiation)	Compute a^n in $O(\log n)$	"power of", "modular exponentiation", "fast power"	Large exponents; always with mod	$a^n = (a^{(n/2)})^2$ if n even; $a^n = a * a^{(n-1)}$ if n odd	$O(\log n)$
Math	Combinatorics nCr	Choose r items from n ; $nCr = n! / (r! (n-r)!)$	"how many ways", "choose r from n ", "count selections"	Counting combinations	Precompute factorials and inverse factorials mod p ; $nCr = \text{fact}[n] * \text{inv_fact}[r] * \text{inv_fact}[n-r] \% p$	$O(n)$ precomp $O(1)$ query
Math	Inclusion-Exclusion	Count elements in union of sets	"count numbers divisible by at least one of", "at least one of"	Union of constraint-satisfying sets		AUBUC

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach	Pros
		using alternating add/subtract	least one condition"			
Math	Digit Decomposition	Process a number digit by digit	"digit sum", "count digits satisfying property", "manipulate digits"	Problems involving individual digits	$n\%10$ gives last digit; $n/10$ removes last digit; iterate until $n=0$	Simple lc
Math	Geometry Basics	Area, distance, collinearity, cross product	"area of triangle", "points on line", "collinear"	Basic geometry problems	Cross product for area/collinearity; dot product for angles; Pythagorean for distance	$O(1)$ for n

SECTION 20: JAVA DATA STRUCTURE INTERVIEW PATTERNS

Java's built-in data structures are highly optimized and frequently appear in interview solutions. Knowing *which structure to reach for* is as important as knowing the algorithm.

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach
Java DS	TreeMap	Sorted map with $O(\log n)$ operations; supports floorKey, ceilingKey	"sorted by key", "find nearest smaller/larger key", "range of keys"	When you need a map AND sorted order	TreeMap.floorKey(k) gives ceilingKey(k) gives smallest
Java DS	TreeSet	Sorted set with $O(\log n)$ operations; supports floor, ceiling	"sorted unique elements", "nearest element", "predecessor/successor"	Ordered set with neighbor queries	TreeSet.floor(x) and ceiling without linear scan
Java DS	PriorityQueue	Min-heap by default; max-heap with reversed comparator	"top k", "always process smallest/largest", "running median"	Greedy problems needing dynamic min/max	PriorityQueue<>((a,b)->b-a) gives minimum always
Java DS	Deque / ArrayDeque	Double-ended queue; $O(1)$ addFirst, addLast, removeFirst, removeLast	"sliding window maximum", "stack + queue hybrid", "BFS + DFS"	Monotonic window, stack simulation, BFS	Use as stack (push/pop from one end, push one end, pop other)
Java DS	HashMap + Frequency	Count frequencies of elements	"frequency", "count", "most common", "group by"	Any frequency counting	getOrDefault pattern; merge
Java DS	Custom Comparator	Override compare() for non-	"sort by multiple fields", "sort by custom rule", "largest number"	When natural ordering	Comparator.comparing(key) or lambda

Topic	Pattern	Information	Recognition Hint	When To Use	Core Approach
		standard ordering		isn't what's needed	
Java DS	Stack via Deque	Use Deque as stack (push to front, pop from front)	Stack-based algorithms	Any stack problem	ArrayDeque as stack is fast

SECTION 21: INTERVIEW META PATTERNS

If Problem Says → Think Pattern

This is your primary diagnostic tool. Before writing any code, scan the problem for these keywords and immediately narrow your search space.

Problem Says / Contains	Think Pattern	Why
Sorted array	Two Pointer / Binary Search	Sorted order enables elimination of halves
Substring / contiguous subarray	Sliding Window / Prefix Sum	Contiguous sequence → window or prefix trick
All subsets / all permutations / generate all	Backtracking	Exponential state space → recursive exploration
Top K / K largest / K smallest	Heap	Dynamic best-K = heap of size K
Range query	Prefix Sum / Segment Tree / Fenwick	Precomputation for repeated range queries
Shortest path (unweighted)	BFS	BFS guarantees shortest path
Shortest path (weighted, non-negative)	Dijkstra	Greedy relaxation works without negative edges
Continuous subarray / minimum length window	Sliding Window	Variable window expand-contract
Overlapping intervals / meetings / schedule	Sort + Interval Pattern	Sorting by start/end reveals structure
Cycle detection	Fast-Slow Pointer / Graph DFS (gray-white-black)	Floyd's detects cycles in $O(n)$ $O(1)$
Maximize ways / count paths / how many ways	DP	Overlapping subproblems + optimal substructure
Next greater element / previous smaller	Monotonic Stack	Stack maintains nearest unsatisfied element
String matching / find pattern in text	KMP / Z / Rabin-Karp	Linear-time pattern search
Palindrome	Expand Around Center / Manacher / DP	Palindrome has center symmetry
Frequency + order maintained	HashMap + TreeMap / HashMap + Heap	Count frequencies; order by frequency or value

Problem Says / Contains	Think Pattern	Why
Prerequisites / dependency order	Topological Sort	DAG ordering = topological sort
Grid movement / shortest path in grid	BFS / DFS / DP	Grid = implicit graph
Repeated queries on static data	Sparse Table / Prefix / Segment Tree	Precompute answers
Binary decisions / yes-no per element	DP / Bitmask DP	Decision tree with overlapping states
Find missing / single / duplicate	XOR Bit Trick / Math	XOR cancels pairs; sum formula finds missing
Merge / union two sorted lists	Merge Pattern / Heap	Two pointers or heap for sorted merge
At most K / exactly K	Sliding Window (at most k) minus (at most k-1) trick	"Exactly k = at most k - at most (k-1)"
Connected components / union	Union Find / BFS DFS	DSU for dynamic; BFS/DFS for static
Maximize minimum / minimize maximum	Binary Search on Answer	Monotonic checker + binary search
Values very large (10^9) but count small (10^5)	Coordinate Compression	Compress values to small indices
Find LCA	Binary Lifting / Euler Tour + RMQ	Precompute ancestor jumps
Infinite series / recurrence	Matrix Exponentiation / DP	Fast computation of nth term
Multiple strings / dictionary queries	Trie	Prefix sharing + $O(L)$ lookup

Problem Phrasing to Pattern Mapping

Phrasing Clue	Pattern Family
"minimum/maximum possible value"	Binary Search on Answer
"number of ways to..."	DP (counting)
"can we / is it possible to..."	DP (boolean) / Greedy
"find the longest/shortest..."	DP / Sliding Window
"find all..."	Backtracking / DFS
"in $O(\log n)$ "	Binary Search / Balanced BST / Heap
"without extra space"	Two Pointer / In-place / Morris
"stream of numbers"	Heap / Running Statistics
"graph / network / connections"	Graph Traversal
"matrix / grid"	BFS/DFS on Grid / Grid DP
"parentheses / brackets"	Stack
"merge two sorted..."	Two Pointer Merge
"maximum subarray"	Kadane's (DP)
"k-th element"	Heap / Quickselect / BST
"anagram / scrambled"	Frequency Map / Sorting
"prefix / suffix"	Prefix DS / KMP / Trie
"range update then query"	Difference Array / Lazy Segment Tree

SECTION 22: LEARNING ORDER FOR BEGINNER

The learning order below follows a principle: **each phase gives you the tools needed to understand the next phase**. Jumping ahead is counterproductive — DP on trees requires knowing both trees and DP; graph algorithms require BFS/DFS intuition from trees.

Phase 1: Foundations (Weeks 1-3)

Topics: Arrays, Hashing, Two Pointers, Sliding Window

Why this first: These are the tools you use in 60% of interview problems. Hashing gives you $O(1)$ lookup. Two pointers eliminate brute force. Sliding window handles contiguous sequence problems. Before learning anything else, you should be able to solve most easy problems and many medium problems using just these four tools.

How to approach: Solve at least 5 problems per pattern before moving on. The goal isn't to memorize — it's to *feel* when to reach for each tool.

Phase 2: Linear Structures (Weeks 4-6)

Topics: Binary Search, Stack, Queue, Linked List

Why this order: Binary search requires the mental model of monotonicity — which you start to see after working with sorted arrays. Stacks and queues are the backbone of tree and graph traversal. Linked lists teach pointer manipulation. These structures are the building blocks of Phase 3.

Phase 3: Hierarchical Structures (Weeks 7-9)

Topics: Trees, Recursion, Backtracking

Why this order: Trees are recursive by nature — learning recursion and trees together reinforces both. Backtracking is recursion with an undo step. Understanding DFS on trees makes graph DFS trivial later. Spend extra time here; tree problems are extremely common in FAANG interviews.

Phase 4: Optimization & Graphs (Weeks 10-13)

Topics: Heap, Greedy, Graph

Why this order: Heap is the data structure that makes greedy algorithms efficient. Greedy and graph algorithms both require a solid understanding of "which step to take next" — often answered by a priority queue. Graph algorithms (Dijkstra, topological sort, MST) combine all previous knowledge.

Phase 5: Dynamic Programming (Weeks 14-17)

Topics: DP (Memoization → Tabulation → Space Optimization → DP Families)

Why this last among core topics: DP is the hardest pattern to internalize because it requires recognizing *structure* in a problem rather than applying a template. The more patterns you know before learning DP, the more clearly you'll see where DP is needed versus where greedy or recursion suffices. Don't rush this phase — spend a full week on each DP family.

Phase 6: Advanced Topics (Weeks 18-24)

Topics: Strings (KMP, Z, Manacher), Math/Number Theory, Range Query DS (Segment Tree, Fenwick), Tries, Bit Manipulation, Advanced Graph (Bridges, SCC, 2-SAT)

Why this last: These are powerful tools for hard problems. Most FAANG interviews rarely go beyond Phase 5 for standard roles. Phase 6 is for competitive programming rounds, senior positions, or "stretch" problems. However, Tries and Bit Manipulation come up more often than the others and should be learned before the rest of Phase 6.

SECTION 23: TOP 20 INTERVIEW PATTERNS

Ranked by frequency and importance across FAANG and top-tier interviews:

Rank	Pattern	One-Line Explanation
1	Sliding Window	Maintains a window over an array/string to find optimal subarrays in $O(n)$
2	Two Pointers	Uses two indices moving in tandem to reduce $O(n^2)$ to $O(n)$ on sorted/structured arrays
3	BFS (Graphs & Trees)	Level-by-level traversal guaranteeing shortest path in unweighted graphs
4	DFS + Backtracking	Exhaustive exploration of all possibilities with pruning for efficiency
5	Dynamic Programming	Solves optimization and counting problems by caching overlapping subproblems
6	Binary Search	Eliminates half the search space per step using a monotonic predicate
7	Prefix Sum	Enables $O(1)$ range sum queries after $O(n)$ preprocessing
8	Hash Map	Converts $O(n)$ lookup to $O(1)$, enabling linear-time solutions to pair/frequency problems
9	Heap (Priority Queue)	Dynamically maintains min/max for greedy selection and top-K problems
10	Tree DFS (Recursion)	Bottom-up or top-down tree computation using postorder/preorder traversal
11	Union Find	Groups nodes dynamically; detects cycles and connected components in near $O(1)$
12	Topological Sort	Orders a DAG for dependency resolution; detects cycles in directed graphs
13	Monotonic Stack	Finds next/previous greater/smaller element in $O(n)$ using a maintained-order stack
14	Greedy	Makes locally optimal choices that lead to globally optimal solutions
15	Binary Search on Answer	Converts "find the optimal value" into "can this value work?" then binary searches
16	Interval Merge/Sweep	Handles overlapping ranges by sorting and sweeping with events

Rank	Pattern	One-Line Explanation
17	Dijkstra	Finds shortest path in weighted non-negative graphs using a priority queue
18	Trie	Enables $O(L)$ prefix queries on large string dictionaries
19	Bit Manipulation (XOR)	Solves uniqueness/parity problems with $O(1)$ space using XOR properties
20	Segment Tree / Fenwick	Handles dynamic range queries and updates in $O(\log n)$

SECTION 24: PATTERN DEPENDENCY GRAPH

Understanding which patterns build on others helps you plan your learning path and debug your understanding when stuck.

FOUNDATION LAYER (no prerequisites):

- Arrays & Strings
- Basic Math (mod, gcd)
- Recursion (base case + recursive case)
- HashMaps / HashSets

LAYER 1 (requires Foundation):

- Two Pointers ————— requires: arrays
- Sliding Window ————— requires: arrays, hashmaps
- Prefix Sum ————— requires: arrays
- Binary Search ————— requires: arrays, monotonicity intuition
- Linked List ————— requires: pointer/reference model

LAYER 2 (requires Layer 1):

- Stack ————— requires: arrays, recursion intuition
- Queue / Deque ————— requires: arrays
- Sorting-based patterns — requires: arrays, binary search
- Interval Patterns ————— requires: sorting, two pointers

LAYER 3 (requires Layer 2):

- Trees (DFS/BFS/DP on Trees) — requires: recursion, stack, queue
- Backtracking ————— requires: recursion, decision tree mental model
- Heap / Priority Queue ————— requires: arrays, comparators

LAYER 4 (requires Layer 3):

- Graph Algorithms (BFS/DFS) — requires: trees, queue, stack
- Greedy ————— requires: sorting, heap
- Union Find ————— requires: arrays
- Topological Sort ————— requires: graph BFS/DFS

LAYER 5 (requires Layer 4):

- Dynamic Programming ————— requires: recursion, subproblem thinking, all of Layer 1-4
- Dijkstra ————— requires: graph BFS, heap
- Monotonic Stack/Queue ————— requires: stack, queue, arrays

LAYER 6 (requires Layer 5):

- Segment Tree / Fenwick ————— requires: prefix sum, divide and conquer
- Trie ————— requires: trees, hashmaps, recursion
- Bitmask DP ————— requires: bit manipulation, DP
- Advanced Graph (SCC, Bridges) — requires: graph DFS, Union Find

LAYER 7 (requires Layer 6):

KMP / Z Algorithm	requires: prefix function understanding, DP intuition
Suffix Array	requires: sorting, binary search, string fundamentals
Aho-Corasick	requires: Trie, KMP
Sparse Table	requires: prefix structures, binary lifting
Matrix Exponentiation	requires: DP recurrence, fast power

Key insight: DP sits at Layer 5 because it requires you to recognize subproblem structure — which only becomes natural after you've seen how recursion, trees, graphs, and greedy all break problems into smaller pieces. Don't skip directly to DP.

SECTION 25: REVISION STRATEGY

Weekly Pattern Revision System

Effective revision is not re-reading this handbook. It is **active recall** — looking at a problem and identifying the pattern before reading the solution.

Week 1-2: Learn and solve 5 problems per pattern (Phases 1-2). **Week 3-4:** Learn and solve 5 problems per pattern (Phases 3-4). **Week 5-6:** DP families — one family per day; solve 3 problems per family. **Week 7 onwards:** Mixed revision — take 3 random problems per day from a problem list. Before solving: identify the pattern. After solving: verify your pattern identification was correct.

How to Identify Patterns from New Problems

Follow this diagnostic sequence when you see a new problem:

Step 1 — Extract constraints: Note n 's size. If $n \leq 20$, think bitmask. If $n \leq 10^5$, $O(n \log n)$ or $O(n)$ needed. If $n \leq 10^3$, $O(n^2)$ is fine.

Step 2 — Extract keywords: Highlight every keyword that maps to a pattern (use the cheat sheet in Section 26).

Step 3 — Ask about structure: Is the data sorted? Is it a graph? Does it have overlapping subproblems? Is it a sequence or a set?

Step 4 — Identify the output type: Is the answer a count, a maximum/minimum, an actual sequence, or a boolean? Counting $\rightarrow$ DP. Maximum $\rightarrow$ DP or Greedy. Sequence $\rightarrow$ Backtracking or DP. Boolean $\rightarrow$ BFS / DP.

Step 5 — Try the simplest matching pattern first: Don't jump to segment trees if a prefix sum works.

How to Avoid Memorizing Solutions

The trap: you solve a problem, read the solution, say "I understand," and then can't solve a similar problem two weeks later. This happens because you memorized the *solution*, not the *pattern*.

The fix: After solving, ask yourself: "What is the key insight that makes this pattern applicable here?" Write it in one sentence. If you can't, you haven't understood the pattern — read more carefully.

The test: Cover the solution. Look only at the problem. Write out what pattern you'd use and why. If you can do this, you know the pattern.

Moving from Beginner to Interview-Ready

Stage 1 (0–50 problems): You're learning to read code and think recursively. Don't worry about optimal solutions. Get something working.

Stage 2 (50–150 problems): Patterns start clicking. You recognize two-pointer and sliding window. Recursion feels natural. Trees become approachable.

Stage 3 (150–300 problems): DP becomes your main challenge. You start seeing overlapping subproblems. Graph problems become manageable.

Stage 4 (300+ problems): You recognize patterns within seconds. You think about multiple approaches before coding. You optimize naturally.

Benchmark: At 200 problems with deliberate pattern practice, you should comfortably solve most medium problems and understand most hard problems. FAANG interviews primarily test medium difficulty with some hard problems — Stage 3 is your target.

SECTION 26: PATTERN RECOGNITION CHEAT SHEET

This is your quick-reference guide. Before solving any problem, scan this sheet.

Clue Phrase / Keyword	Likely Pattern	Why This Pattern Fits
"subarray sum equals k"	Prefix Sum + HashMap	$\text{prefix}[r] - \text{prefix}[l] = k \rightarrow \text{lookup}$ $\text{prefix}[r] - k$ in map
"next greater element"	Monotonic Decreasing Stack	Stack maintains candidates waiting for their "next greater"
"top k frequent"	Min-Heap of size k	Adding to a k-size heap and discarding minimum keeps k largest
"minimum/maximum possible value"	Binary Search on Answer	Monotonic checker on the value space
"all subsets / all permutations"	Backtracking	Exponential state space $\rightarrow$ recursive explore + unchoose
"shortest path unweighted"	BFS	BFS processes nodes in order of distance
"shortest path weighted"	Dijkstra	Greedy relaxation with priority queue
"sort + adjacent comparison"	Sort + Greedy or Two Pointer	Sorting reveals local structure
"palindrome"	Expand Around Center / Manacher / DP	Palindrome has central symmetry
"anagram / permutation of string"	Sliding Window + Frequency Map	Fixed-size window with character counts
"longest increasing subsequence"	DP ($O(n^2)$) / Patience Sort ($O(n \log n)$)	LIS at index i depends on all previous LIS values
"cycle detection"	Fast-Slow Pointer / Graph Coloring	Floyd's algorithm for linked structures
"k-th largest/smallest"	Heap / Quickselect	Heap of size k gives top-k in $O(n \log k)$
"merge intervals"	Sort by start + greedy merge	Sorting makes overlapping intervals adjacent
"count ways to make X"	DP (unbounded/0-1 knapsack)	Counting paths in a decision tree with overlapping states
"range sum query (static)"	Prefix Sum	$O(1)$ query after $O(n)$ build

Clue Phrase / Keyword	Likely Pattern	Why This Pattern Fits
"range sum query (with updates)"	Segment Tree / Fenwick Tree	$O(\log n)$ update + query
"prefix matching / autocomplete"	Trie	Prefix sharing in tree structure
"find missing / single / unpaired"	XOR Bit Trick	XOR cancels pairs; lone element survives
"graph with prerequisites"	Topological Sort	DAG ordering resolves dependencies
"connected components"	Union Find / BFS / DFS	Group nodes by reachability
"grid traversal / island count"	BFS or DFS on 2D grid	Grid = implicit graph with 4 neighbors
"stock buy/sell / at most k transactions"	State Machine DP	States: holding / not holding; transitions = buy/sell
"maximum subarray sum"	Kadane's Algorithm (DP)	Extend or restart at each element
"contains duplicate / two sum"	HashSet / HashMap	$O(1)$ lookup replaces $O(n)$ scan
"inorder of BST gives sorted"	BST Inorder Traversal	BST property: left < node < right
"validate BST"	DFS with min/max bounds	Each node must be in valid range from ancestors
"diameter / longest path in tree"	Postorder DFS	At each node: combine left + right heights
"word ladder / state transition"	BFS on State Graph	States as nodes, transitions as edges, BFS for min steps
"largest rectangle in histogram"	Monotonic Stack	For each bar, find left/right boundary using stack
"sliding window maximum"	Monotonic Deque	Deque maintains decreasing order; front is max
"balance check / height check"	Postorder DFS	Compute height; check
"number of distinct substrings"	Suffix Array / Trie	Enumerate all distinct substrings via suffix structure

Clue Phrase / Keyword	Likely Pattern	Why This Pattern Fits
"dynamic connectivity"	Union Find	Online merging of components
"matrix chain / burst balloons"	Interval DP	Answer for range $[i,j]$ depends on splitting at k
"path in tree not necessarily through root"	Global variable in Postorder DFS	Update global max at each node; don't propagate full path up
"expression evaluation"	Two-Stack or Monotonic Stack	Operator precedence handled by stack ordering
"0-1 knapsack / include-exclude"	0/1 Knapsack DP	$dp[w] = \max/\text{count}$ considering current item
"minimum spanning tree"	Kruskal (sorted edges + DSU) / Prim (heap)	Connect all nodes at minimum total cost
"negative weight edges"	Bellman-Ford	Dijkstra fails with negatives; Bellman-Ford relaxes $V-1$ times
"all pairs shortest path"	Floyd-Warshall	$dp[i][j] = \min$ over all intermediate nodes k
"subset with exact sum"	Subset Sum DP	$dp[j] = \text{can sum } j \text{ be formed from items seen so far}$

End of DSA Pattern Handbook. Use Section 26 as your entry point for every new problem. Use Sections 22–25 to plan and sustain your preparation. Everything else is the reference you build from.